

This code was compiled on WSL in vscode. use the "make" command to compile, and `./execute` to run the program.

To recompile, use "make clean" and then repeat the first steps again.

This code runs without user input. To test additional cases, edit and add them to `pp1_input.txt`.

The program theoretically handles any sized file, so you can throw additional operations in the lines below the pre-loaded test cases.

Logic of ADD: The ADD operation adds two passed hex values together, stores that value in a variable, and then outputs the value in that variable.

ADD 0x5 0x5 will output 0xA

Logic of SUB: The SUB operation subtracts two passed hex values together, stores that value in a variable, and then outputs the value in that variable.

SUB 0x9 0x8 will output 0x1

Logic of AND: The AND operation performs a bitwise and on two passed hex values, stores that value in a variable, and then outputs the value in that variable.

AND 0xA 0x6 will output 0x2

Logic of OR: The OR operation performs a bitwise or on two passed hex values, stores that value in a variable, and then displays the value in that variable.

OR 0x3 0x8 will output 0xB

Logic of XOR: The XOR operation performs a bitwise xor on two passed hex values, stores that value in a variable, and then displays the value in that variable.

XOR 0x4 0x7 will output 0x3

Logic of NOT: The NOT operation performs a bitwise not on one passed hex value, stores that value in a variable, and then outputs the value in the variable.

NOT 0x5 will output 0xA

Logic of LSL and LSR: Two values are passed into the operation: the hex value, and the int corresponding to the number of shifts wanted. If the number of shifts is greater than 0, but less than 32, the program performs bitwise shifts on b corresponding to i, stores the shifted value in a variable, and displays the value in that variable. If the shift number is greater than 32, an error of <shift value exceeds bit size> is output.

LSL 0x3 2 will output 0xC

Logic of EQ: Two hex values are passed into the function. If they are equal, true is displayed. If they are not equal, false is displayed. A third edge case, if neither trigger is also there, which displays <error with EQ>. This should theoretically not be possible, and is there for debugging.

LSR 0x8 2 will output 0x2

Logic of GT: Two hex values are passed into the function. If b1 is greater than b2, true is displayed. If b1 is less than or equal to b2, false is displayed. A third edge case, if neither trigger is also there, which displays <error with GT>. This should theoretically not be possible, and is there for debugging.

GT 0xF 0xE will output <true>

Logic of LT: Two hex values are passed into the function. If b1 is less than b2, true is displayed. If b1 is greater than or equal to b2, false is displayed. A third edge case,

if neither trigger is also there, which displays <error with LT>. This should theoretically not be possible, and is there for debugging.

LT 0xF 0xF will output false

There are a few main invalid cases: <Unsupported operation>, <Shift value exceeds bit size>, <negative shift count>, and <invalid operand count>

<Unsupported operation>: when anything stored in arrStr is passed through operationSet, if it does not meet any of the conditionals, it means it is an operation that does not exist.

<Shift value exceeds bit size>: If the shift value is detected to be bigger than 31, it is automatically flagged as being outside of the bit size and prevented from executing.

<negative shift count> If the shift count is determined to be negative, we prevent shifting from happening. This is due to the fact that shifts cannot compute negative shift values.

<invalid operand count>: There are two cases here: there are too few operands, or too many. If an single operand is passed and the operation is not NOT, it is invalid.

If anything has more than two operands, it is invalid as well.

Extra credit:

To perform multiplication, you would use LSL. For division, you would use LSR. Shifting a binary number left by 1 is the same as multiplying by 2, and shifting to the right by 1 is the same as dividing by 2. We can either maintain the same LSL/LSR code we have, or we can make a new operation called MULTIPLY and DIVIDE. In this case, we would have the inputs be: OPERATION 0x____ #N. In this case, we would have MULTIPLY 0x____ #N, where we perform a bitwise LSL for however much we want to multiply by. Important, however, is that we would either need to make if statements for all powers of 2 within the limit, or implement logic that can determine the Number of bits to shift by. If we wanted to multiply 0x0056 by 8, for example, we want to perform LSL 0x0056 #3; If we did #8, it would be incorrect.

For division, it is very similar, except with LSR. The inputs would be OPERATION 0x____ #N, or DIVIDE 0x____ #N, where n is not the number we want to divide by, but how many bits to the right to shift by. We would once again need to either make if statements covering all $1/(2^n)$, or implement logic to determine it. For example, if you want to divide 0x800c by 4, we dont want DIVIDE 0x800C # $\frac{1}{4}$, or DIVIDE 0X800C #4, we want DIVIDE 0X800C #2 (which is the same as LSR 0X800C #2).